SHELLMATES CLUB

BINARY EXPLOITATION & REVERSE ENGINEERING BASICS

/*WHERE THERE'S A SHELL, THERE'S A WAY*/



Reverse Engineering

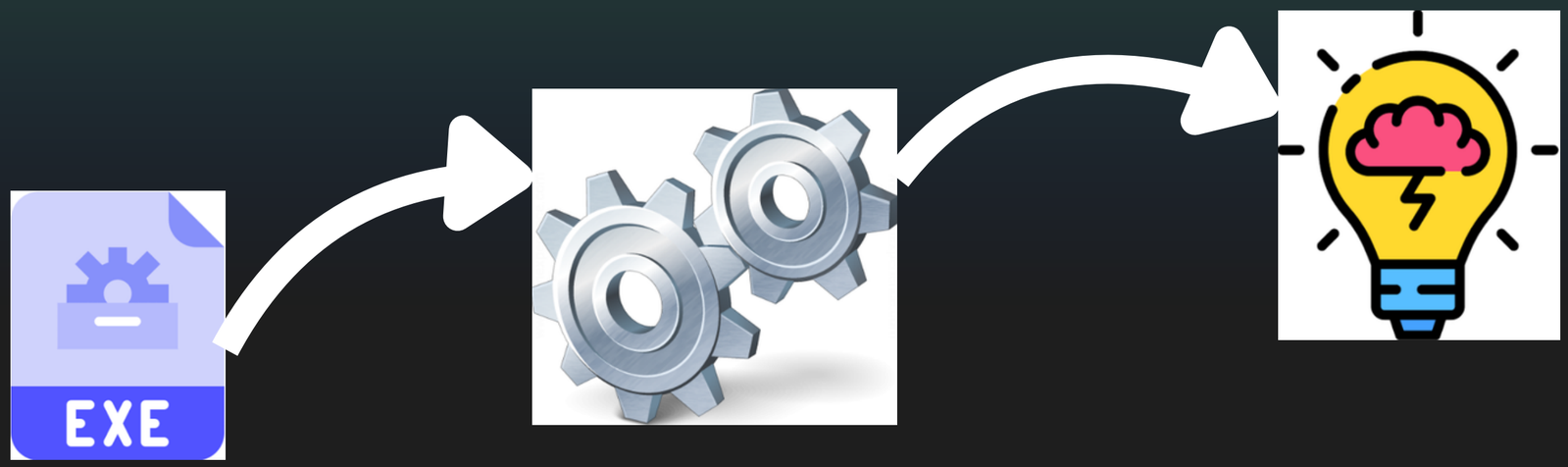
SHELLMATES CLUB

1. Reverse Engineering Basics
2. Getting Started
3. Targets & Binary Formats
4. Assembly Language
5. Static & Dynamic Analysis
6. Examples & Q&A

/*WHERE THERE'S A SHELL, THERE'S A WAY*/

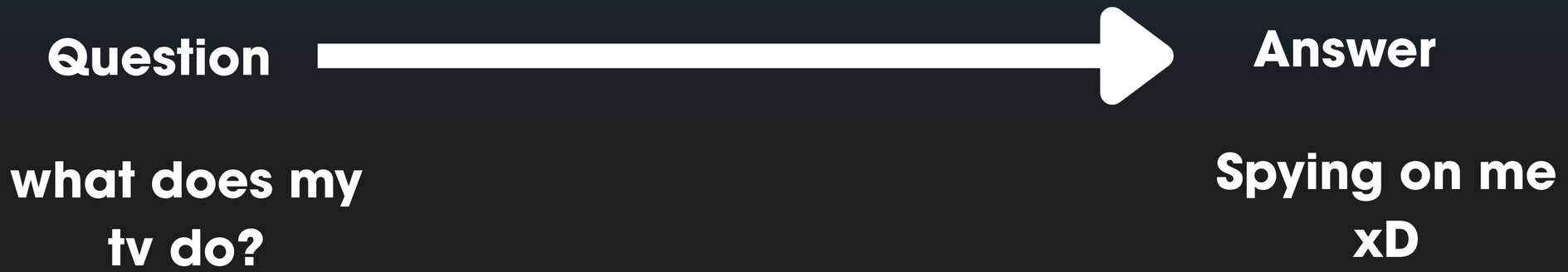
WHAT IS REVERSE ENGINEERING?

Reverse engineering is the process of examining an application to understand how it works without access to its original source code or design.



WHAT IS REVERSE ENGINEERING?

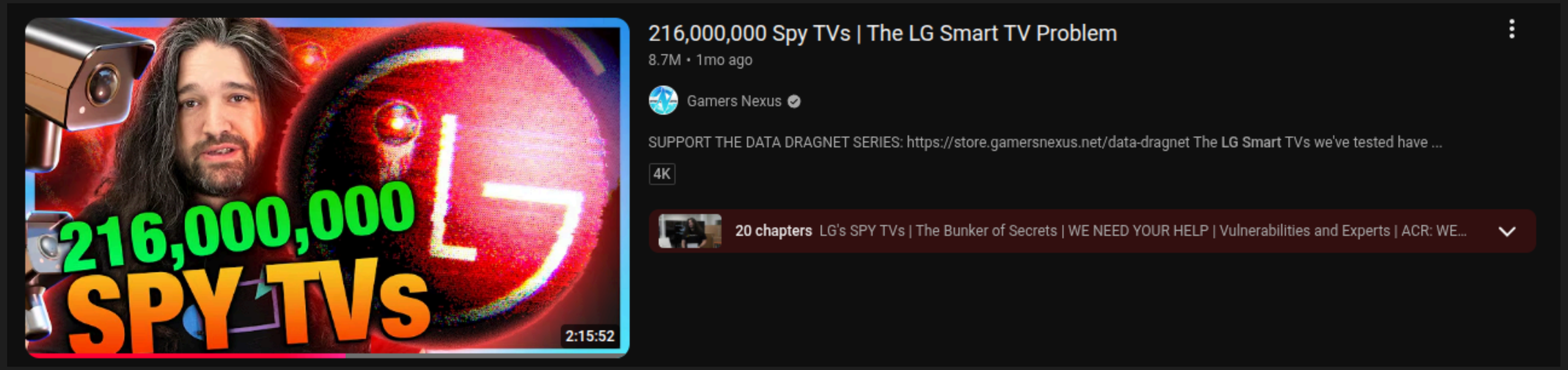
Or we can define it by,
we can define it as starting with a question about something we know absolutely nothing about, and working backward through reverse engineering to find the answer.



EXAMPLES

SHELLMATES CLUB

REAL-WORLD VULNERABILITY DEMONSTRATIONS



/*WHERE THERE'S A SHELL, THERE'S A WAY*/

Why It Matters

Why It Matters
Fixing Bugs
Deep understanding of CS principles and low-level systems
Equips both attackers and defenders with crucial technical skills.
Builds a strong mental model and encourages flexible thinking.



SHELLMATES
CLUB

HOW TO GET STARTED

MOBILE

APPLICATIONS

HARDWARE

DEPENDS

GAME

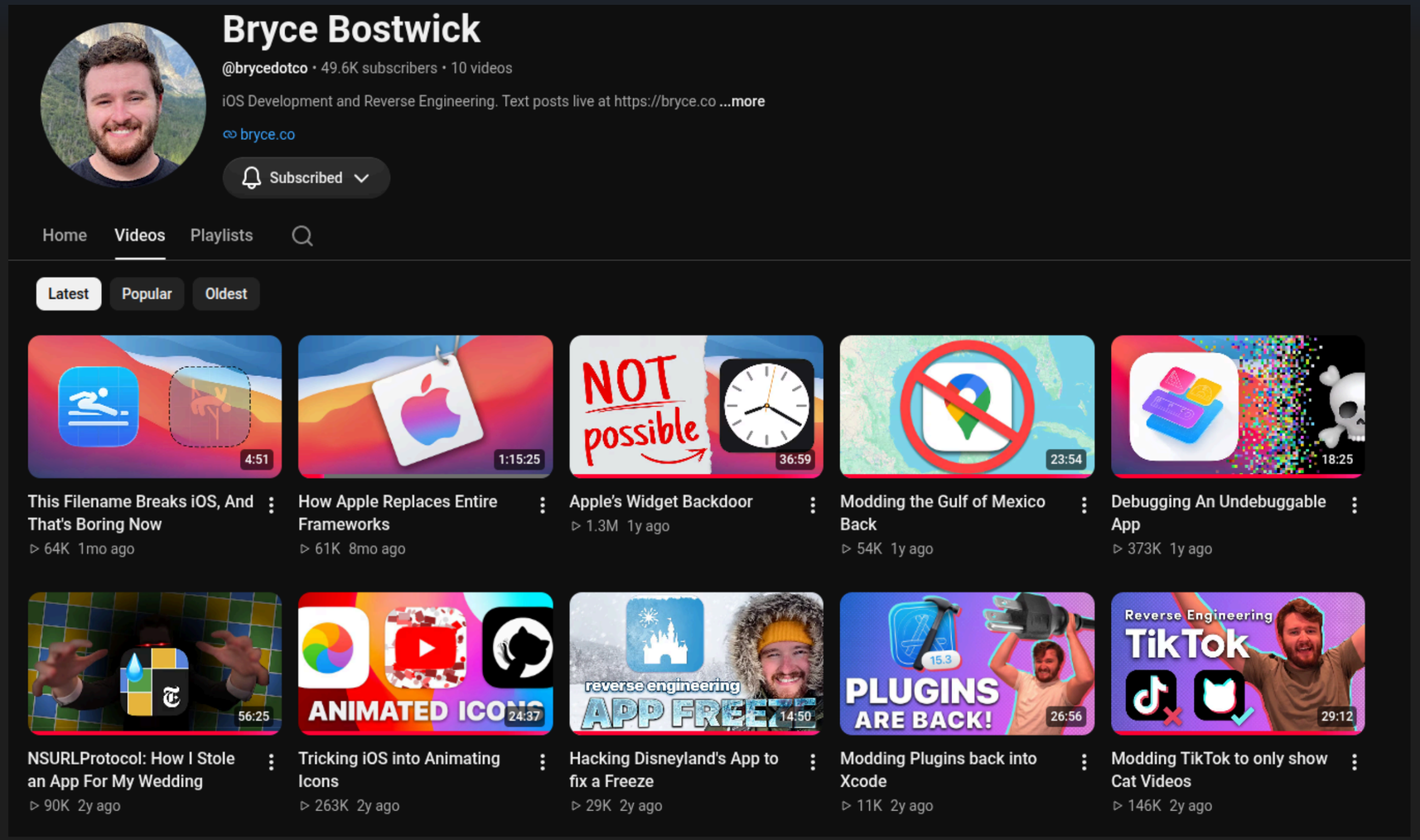
MALWARE

/*WHERE THERE'S A SHELL, THERE'S A WAY*/

EXAMPLES

SHELLMATES CLUB

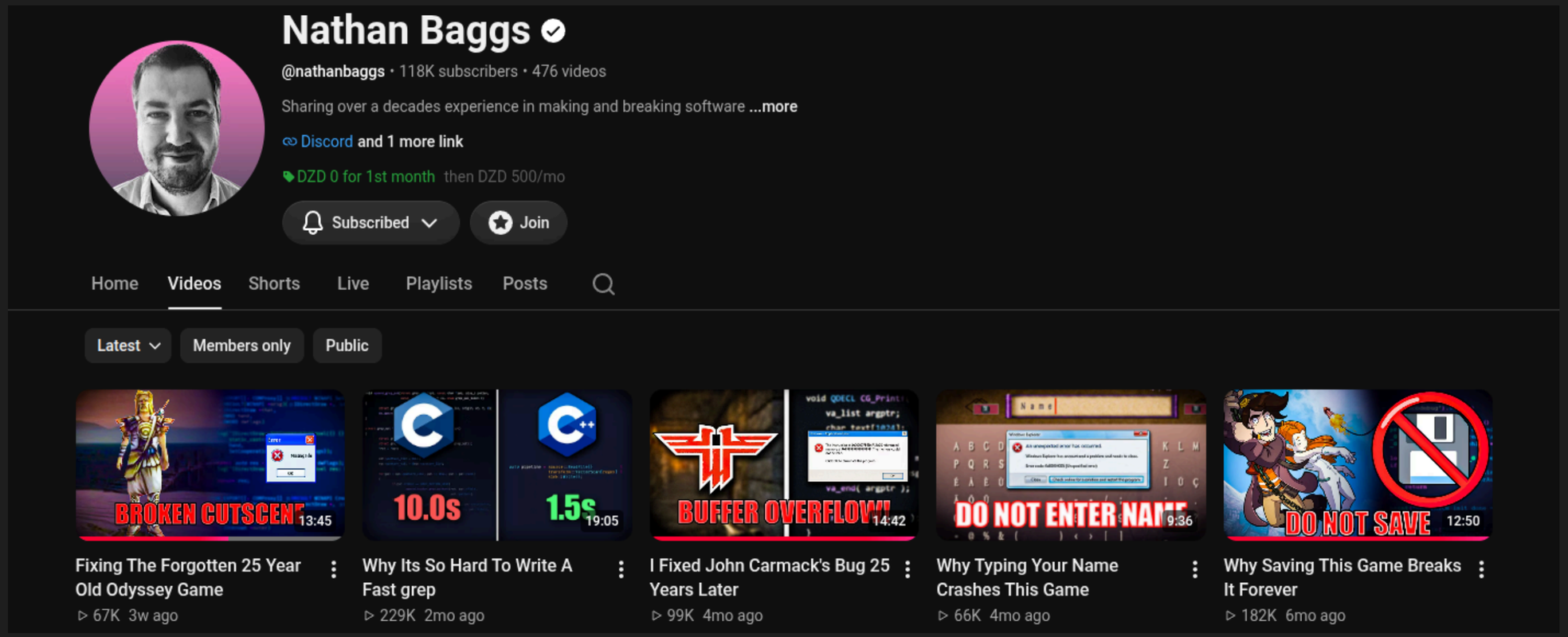
REAL-WORLD VULNERABILITY DEMONSTRATIONS



EXAMPLES

SHELLMATES CLUB

REAL-WORLD VULNERABILITY DEMONSTRATIONS





SHELLMATES
CLUB

HOW TO GET STARTED

MOBILE

APPLICATIONS

HARDWARE

DEPENDS

GAME

MALWARE

/*WHERE THERE'S A SHELL, THERE'S A WAY*/



SHELLMATES
CLUB

HOW TO GET STARTED

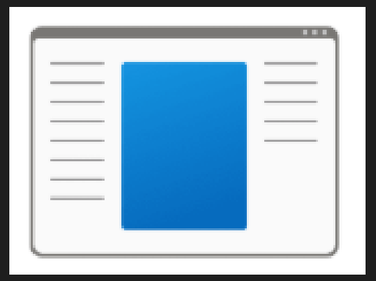
APPLICATIONS

/*WHERE THERE’S A SHELL, THERE’S A WAY*/

SHELLMATES
CLUB

FUNDAMENTALS & BASICS

- Essential tools for debugging and analysis
- Assembly language
- Program Env



/*WHERE THERE'S A SHELL, THERE'S A WAY*/

SHELLMATES CLUB

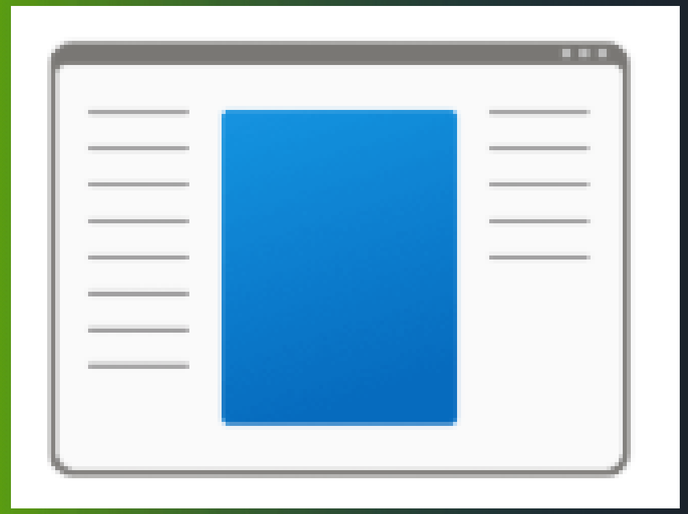
TARGET



/*WHERE THERE'S A SHELL, THERE'S A WAY*/

SHELLMATES CLUB

TARGET



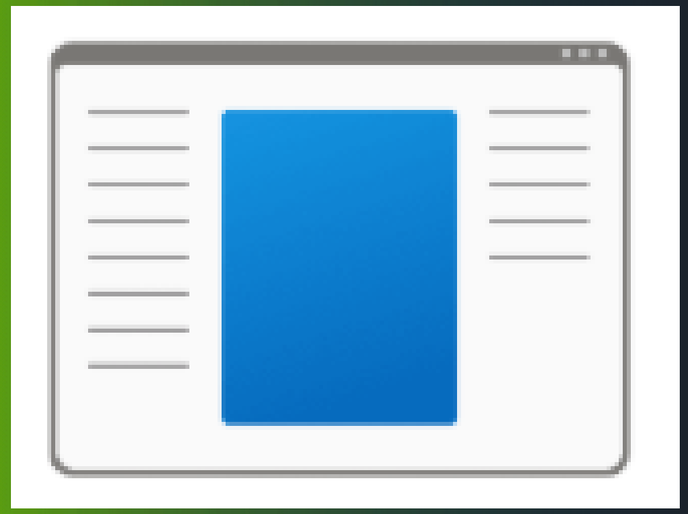
TYPE:

READABLE CONTENT (STRINGS):

BEHAVIOR:

SHELLMATES CLUB

TARGET



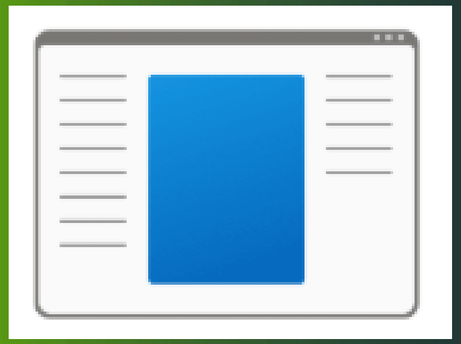
TYPE: ELF BINARY

READABLE CONTENT (STRINGS):
TWO STATES OF THE APP

BEHAVIOR: UNDERSTOOD

SHELLMATES CLUB

Assembly language



READER



File Edit View Workspace Extras Help

Hex editor

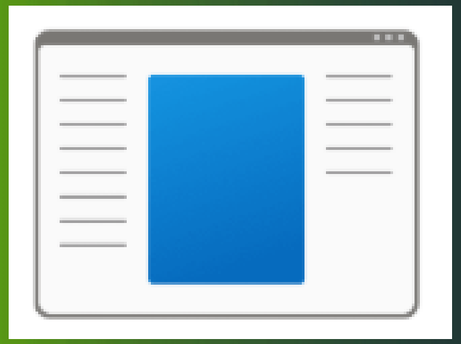
```
Address    00 01 02 03 04 05 06 07  08 09 0A 0B 0C 0D 0E 0F  ASCII

00000000:  7F 45 4C 46 02 01 01 00  00 00 00 00 00 00 00 00  .ELF............
00000010:  03 00 3E 00 01 00 00 00  E0 10 00 00 00 00 00 00  ..>.............
00000020:  40 00 00 00 00 00 00 00  80 37 00 00 00 00 00 00  @........7......
00000030:  00 00 00 00 40 00 38 00  0D 00 40 00 1F 00 1E 00  ....@.8...@.....
00000040:  06 00 00 00 04 00 00 00  40 00 00 00 00 00 00 00  ........@.......
00000050:  40 00 00 00 00 00 00 00  40 00 00 00 00 00 00 00  @.......@.......
00000060:  D8 02 00 00 00 00 00 00  D8 02 00 00 00 00 00 00  ................
00000070:  08 00 00 00 00 00 00 00  03 00 00 00 04 00 00 00  ................
00000080:  18 03 00 00 00 00 00 00  18 03 00 00 00 00 00 00  ................
00000090:  18 03 00 00 00 00 00 00  1C 00 00 00 00 00 00 00  ................
000000A0:  1C 00 00 00 00 00 00 00  01 00 00 00 00 00 00 00  ................
000000B0:  01 00 00 00 04 00 00 00  00 00 00 00 00 00 00 00  ................
000000C0:  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
000000D0:  88 07 00 00 00 00 00 00  88 07 00 00 00 00 00 00  ................
000000E0:  00 10 00 00 00 00 00 00  01 00 00 00 05 00 00 00  ................
000000F0:  00 10 00 00 00 00 00 00  00 10 00 00 00 00 00 00  ................
00000100:  00 10 00 00 00 00 00 00  85 02 00 00 00 00 00 00  ................
00000110:  85 02 00 00 00 00 00 00  00 10 00 00 00 00 00 00  ................
00000120:  01 00 00 00 04 00 00 00  00 20 00 00 00 00 00 00  ......... ......
00000130:  00 20 00 00 00 00 00 00  00 20 00 00 00 00 00 00  . ....... ......
00000140:  D4 01 00 00 00 00 00 00  D4 01 00 00 00 00 00 00  ................
00000150:  00 10 00 00 00 00 00 00  01 00 00 00 06 00 00 00  ................
00000160:  98 2D 00 00 00 00 00 00  98 3D 00 00 00 00 00 00  .-.......=......
00000170:  98 3D 00 00 00 00 00 00  78 02 00 00 00 00 00 00  .=......x.......
00000180:  88 02 00 00 00 00 00 00  00 10 00 00 00 00 00 00  ................
00000190:  02 00 00 00 06 00 00 00  A8 2D 00 00 00 00 00 00  .........-......
000001A0:  A8 3D 00 00 00 00 00 00  A8 3D 00 00 00 00 00 00  .=.......=......
000001B0:  F0 01 00 00 00 00 00 00  F0 01 00 00 00 00 00 00  ................
000001C0:  08 00 00 00 00 00 00 00  04 00 00 00 04 00 00 00  ................
000001D0:  38 03 00 00 00 00 00 00  38 03 00 00 00 00 00 00  8.......8.......
000001E0:  38 03 00 00 00 00 00 00  30 00 00 00 00 00 00 00  8.......0.......
000001F0:  30 00 00 00 00 00 00 00  08 00 00 00 00 00 00 00  0...............
00000200:  04 00 00 00 04 00 00 00  68 03 00 00 00 00 00 00  ........h.......
00000210:  68 03 00 00 00 00 00 00  68 03 00 00 00 00 00 00  h.......h.......
00000220:  44 00 00 00 00 00 00 00  44 00 00 00 00 00 00 00  D.......D.......
```

/*WHERE THERE'S A SHELL, THERE'S A WAY*/

SHELLMATES CLUB

Assembly language



READER

objdump

```
1211:              call   10c0 <fflush@plt>
1216:              lea    rax,[rbp-0xc]
121a:              mov    rsi,rax
121d:              lea    rax,[rip+0xe79]        # 209d <_IO_stdin_used+0x9d>
1224:              mov    rdi,rax
1227:              mov    eax,0x0
122c:              call   10d0 <__isoc99_scanf@plt>
1231:              mov    eax,DWORD PTR [rbp-0xc]
1234:              cmp    eax,0x1337
1239:        /———— jne    124c <main+0x83>
123b:        |     lea    rax,[rip+0xe5e]        # 20a0 <_IO_stdin_used+0xa0>
1242:        |     mov    rdi,rax
1245:        |     call   1090 <puts@plt>
124a:        |  /— jmp    125b <main+0x92>
124c:        \--|→ lea    rax,[rip+0xe75]        # 20c8 <_IO_stdin_used+0xc8>
1253:           |  mov    rdi,rax
1256:           |  call   1090 <puts@plt>
125b:           \→ mov    eax,0x0
1260:              mov    rdx,QWORD PTR [rbp-0x8]
1264:              sub    rdx,QWORD PTR fs:0x28
126d:           /— je     1274 <main+0xab>
126f:           |  call   10a0 <__stack_chk_fail@plt>
1274:           \→ leave
1275:              ret

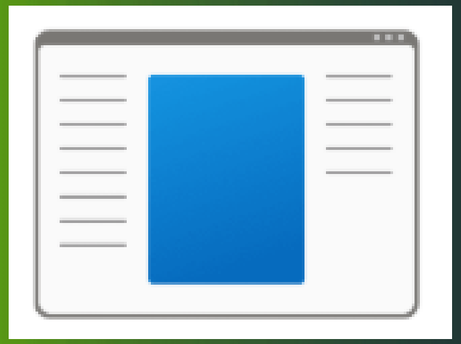
Disassembly of section .fini:
```

/*WHERE THERE'S A SHELL, THERE'S A WAY*/

SHELLMATES CLUB

Assembly language

READER





```
00101207 48 8b 05        MOV        RAX,qword ptr [stdout]
         02 2e 00 00
0010120e 48 89 c7        MOV        RDI,RAX
00101211 e8 aa fe        CALL       <EXTERNAL>::fflush                    int ffl
         ff ff
00101216 48 8d 45 f4     LEA        RAX=>local_14,[RBP + -0xc]
0010121a 48 89 c6        MOV        RSI,RAX
0010121d 48 8d 05        LEA        RAX,[DAT_0010209d]                    = 25h
         79 0e 00 00
00101224 48 89 c7        MOV        RDI=>DAT_0010209d,RAX                 = 25h
00101227 b8 00 00        MOV        EAX,0x0
         00 00
0010122c e8 9f fe        CALL       <EXTERNAL>::__isoc99_scanf            undefin
         ff ff
00101231 8b 45 f4        MOV        EAX,dword ptr [RBP + local_14]
00101234 3d 37 13        CMP        EAX,0x1337
         00 00
00101239 75 11           JNZ        LAB_0010124c
0010123b 48 8d 05        LEA        RAX,[DAT_001020a0]                    = 1Bh
         5e 0e 00 00
00101242 48 89 c7        MOV        RDI=>DAT_001020a0,RAX                 = 1Bh
00101245 e8 46 fe        CALL       <EXTERNAL>::puts                      int put
         ff ff
0010124a eb 0f           JMP        LAB_0010125b

                     LAB_0010124c                                  XREF[1]:    00101239(j
0010124c 48 8d 05        LEA        RAX,[DAT_001020c8]                    = 1Bh
         75 0e 00 00
00101253 48 89 c7        MOV        RDI=>DAT_001020c8,RAX                 = 1Bh
00101256 e8 35 fe        CALL       <EXTERNAL>::puts                      int put
         ff ff

                     LAB_0010125b                                  XREF[1]:    0010124a(j
0010125b b8 00 00        MOV        EAX,0x0
```

/*WHERE THERE'S A SHELL, THERE'S A WAY*/



SHELLMATES
CLUB

STATIC ANALYSIS

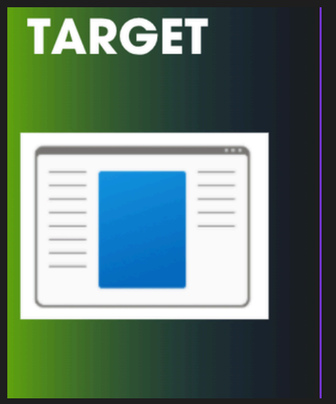
Static analysis examines a program's code or binary without running it to understand how it works.

DYNAMIC ANALYSIS

Dynamic analysis is examining a program while it's running to understand its behavior, monitor its execution, and find potential bugs or vulnerabilities.

SHELLMATES CLUB

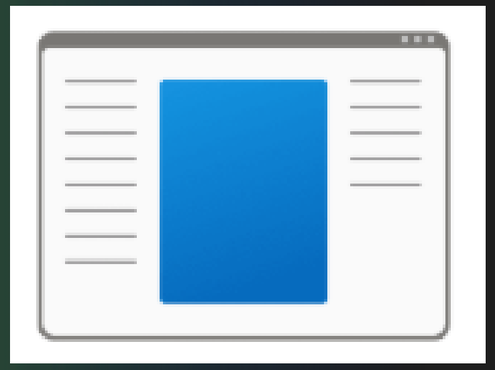
DYNAMIC ANALYSIS



TYPE: ELF BINRAY

READABLE CONTENT (STRINGS): TWO STATES OF APP

BEHAVOUR: UNDERSTOOD



VS



/*WHERE THERE'S A SHELL, THERE'S A WAY*/

SHELLMATES
CLUB

Q&A

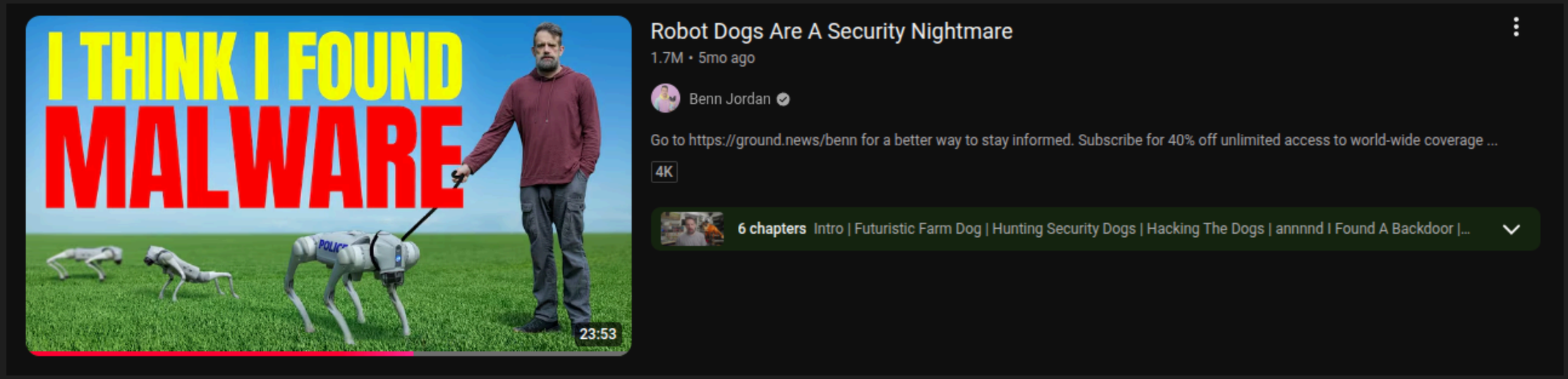
SUMMARY

/*WHERE THERE’S A SHELL, THERE’S A WAY*/

EXAMPLES

SHELLMATES CLUB

REAL-WORLD VULNERABILITY DEMONSTRATIONS



Pwn Basics

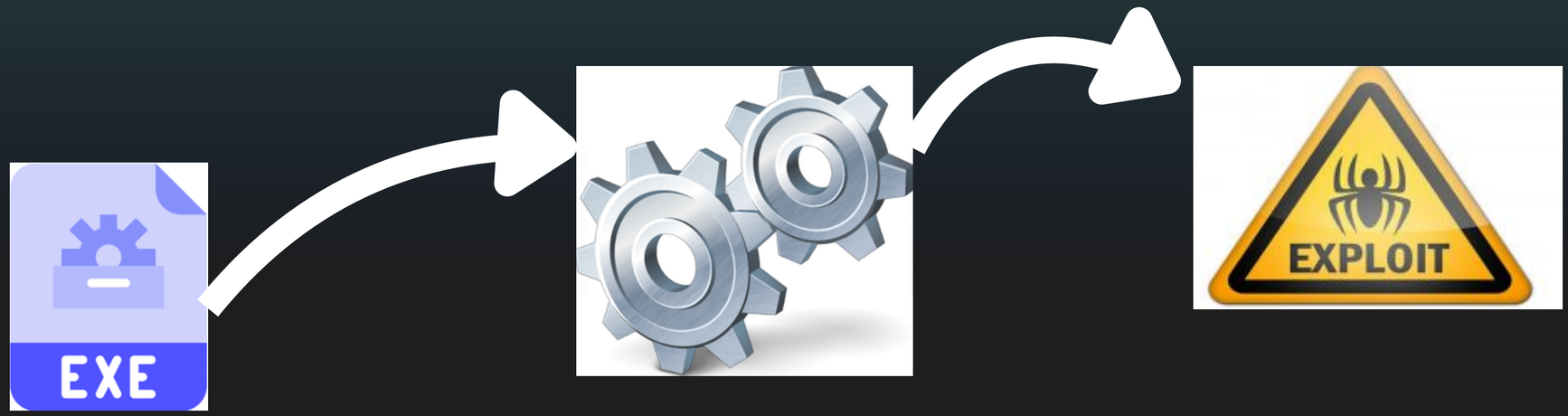
SHELLMATES CLUB

1. Binary Exploitation
2. Bug Classes & Vulnerabilities
3. Buffer Overflows
4. Integer Vulnerabilities
5. Q&A, Resources & Careers

/*WHERE THERE'S A SHELL, THERE'S A WAY*/

WHAT IS BINARY EXPLOITATION?

Binary exploitation is the process of finding bugs in binary programs and turning them into exploits to manipulate their behavior.



WHAT IS A BUG?

A bug is a mistake or flaw in a program that causes unexpected or incorrect behavior, something the programmer didn't intend or account for.

Shopping App
Imagine you're buying a phone online. The app asks you to enter the quantity you want to buy.
What if you enter -5 instead of 5?
If the programmer didn't account for negative numbers and the transaction still goes through, that's a bug.



EXAMPLES

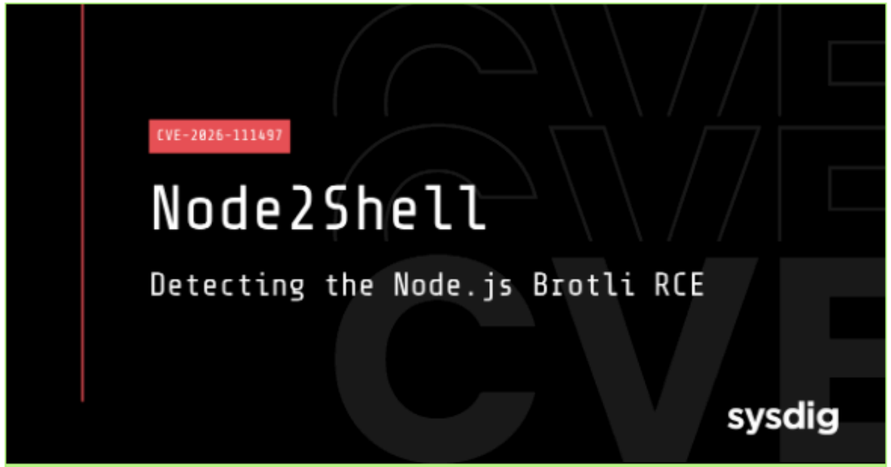
SHELLMATES CLUB

REAL-WORLD VULNERABILITY DEMONSTRATIONS

< BACK TO BLOG

Node2Shell (CVE-2026-111497): Detecting the Node.js Brotli RCE

PUBLISHED BY:
SYSDIG THREAT RESEARCH TEAM



PUBLISHED: OCTOBER 24, 2026

/*WHERE THERE'S A SHELL, THERE'S A WAY*/

SHELLMATES
CLUB

FUNDAMENTALS & BASICS

- Bug Classes
- Reverse Engineering Basics
- Essential tools for debugging and analysis

/*WHERE THERE'S A SHELL, THERE'S A WAY*/



SHELLMATES
CLUB

BUG CLASSES

FORMAT STRING VULNERABILITIES

UNINITIALIZED MEMORY

TYPE CONFUSION

BUFFER OVERFLOWS

RACE CONDITIONS

MEMORY CORRUPTION

INTEGER VULNERABILITIES

/*WHERE THERE'S A SHELL, THERE'S A WAY*/



SHELLMATES
CLUB

BUG CLASSES

BUFFER OVERFLOWS

/*WHERE THERE'S A SHELL, THERE'S A WAY*/



SHELLMATES
CLUB

BUFFER OVERFLOWS

HOW TO TACKLE A BUG CLASS

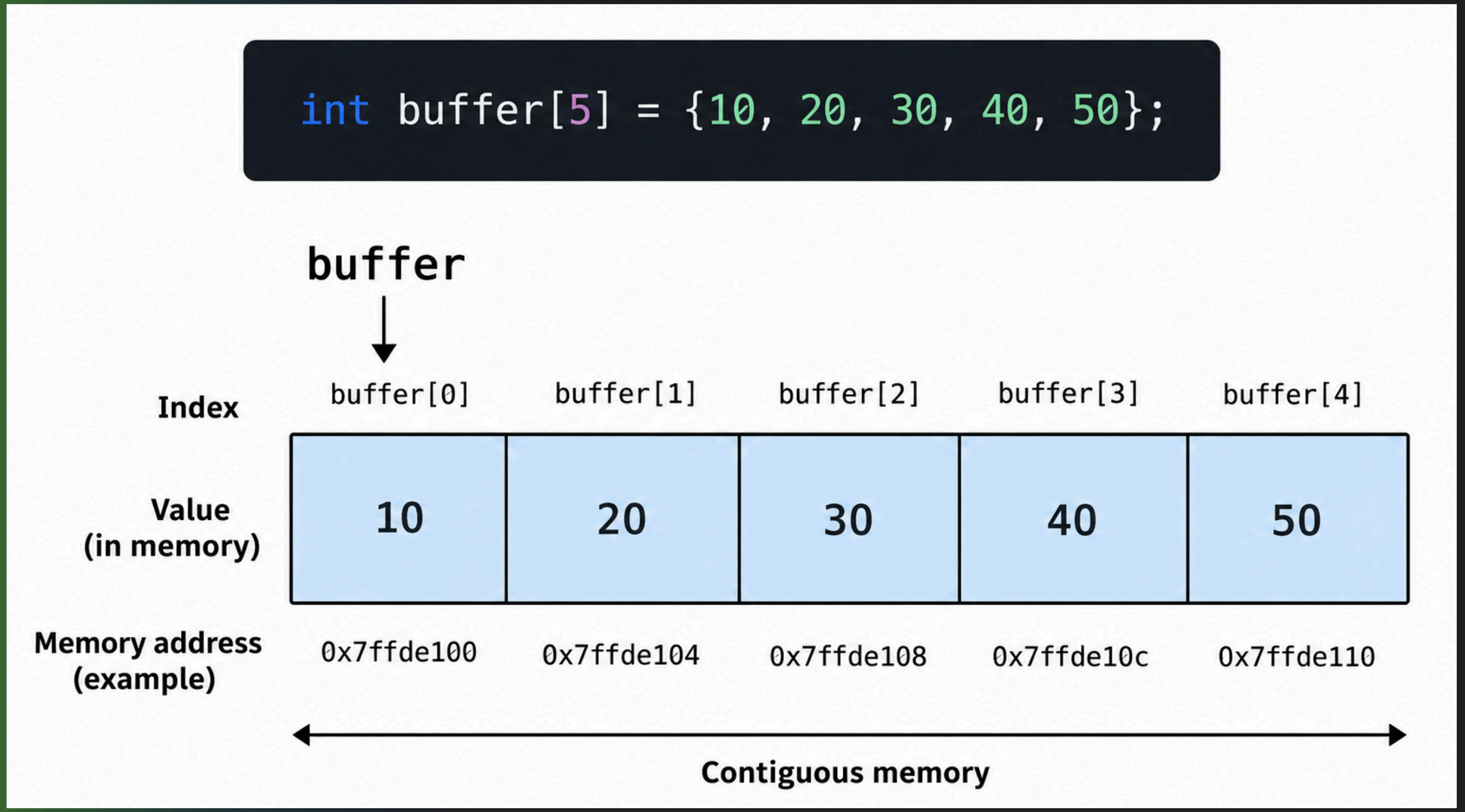
- Understand Data Structures
- Understand Affected Memory
- Understand Use cases
- Practice

/*WHERE THERE'S A SHELL, THERE'S A WAY*/

WHAT'S A BUFFER?

A buffer is simply a region of memory used by a program to temporarily store data. The data is stored in a contiguous manner, meaning the bytes are placed next to each other in memory.

```
int buffer[5] = {10, 20, 30, 40, 50};
```



Understand Data Structures

WHAT'S A BUFFER OVERFLOW?

A buffer overflow occurs when a program writes more data to a buffer than it can hold.

Because buffers are stored alongside other important data in memory, exceeding the buffer's limits can overwrite nearby memory and cause problems.





SHELLMATES
CLUB

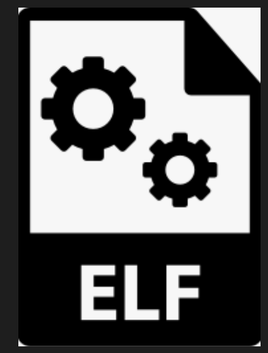
WHERE IS THE DATA STORED?

```
pwndbg> vmmap
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
             Start                End Perm     Size  Offset File (set vmmap-prefer-relpaths on)
    0x555555554000     0x555555555000 r--p     1000       0 exp1
    0x555555555000     0x555555556000 r-xp     1000    1000 exp1
    0x555555556000     0x555555557000 r--p     1000    2000 exp1
    0x555555557000     0x555555558000 r--p     1000    2000 exp1
    0x555555558000     0x555555559000 rw-p     1000    3000 exp1
    0x555555559000     0x55555557a000 rw-p    21000       0 [heap]
    0x7ffff7c00000     0x7ffff7c28000 r--p    28000       0 /usr/lib/x86_64-linux-gnu/libc.so.6
    0x7ffff7c28000     0x7ffff7db1000 r-xp   189000   28000 /usr/lib/x86_64-linux-gnu/libc.so.6
    0x7ffff7db1000     0x7ffff7e00000 r--p    4f000  1b1000 /usr/lib/x86_64-linux-gnu/libc.so.6
    0x7ffff7e00000     0x7ffff7e04000 r--p     4000  1ff000 /usr/lib/x86_64-linux-gnu/libc.so.6
    0x7ffff7e04000     0x7ffff7e06000 rw-p     2000  203000 /usr/lib/x86_64-linux-gnu/libc.so.6
    0x7ffff7e06000     0x7ffff7e13000 rw-p     d000       0 [anon_7ffff7e06]
    0x7ffff7fa3000     0x7ffff7fa6000 rw-p     3000       0 [anon_7ffff7fa3]
    0x7ffff7fbd000     0x7ffff7fbf000 rw-p     2000       0 [anon_7ffff7fbd]
    0x7ffff7fbf000     0x7ffff7fc1000 r--p     2000       0 [vvar]
    0x7ffff7fc1000     0x7ffff7fc3000 r--p     2000       0 [vvar_vclock]
    0x7ffff7fc3000     0x7ffff7fc5000 r-xp     2000       0 [vdso]
    0x7ffff7fc5000     0x7ffff7fc6000 r--p     1000       0 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
    0x7ffff7fc6000     0x7ffff7ff1000 r-xp    2b000    1000 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
    0x7ffff7ff1000     0x7ffff7ffb000 r--p     a000   2c000 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
    0x7ffff7ffb000     0x7ffff7ffd000 r--p     2000   36000 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
    0x7ffff7ffd000     0x7ffff7fff000 rw-p     2000   38000 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
    0x7fffffffd000     0x7ffffffff000 rw-p    22000       0 [stack]
0xffffffffff600000 0xffffffffff601000 --xp     1000       0 [vsyscall]
```

/*WHERE THERE'S A SHELL, THERE'S A WAY*/

SHELLMATES CLUB

BUG



pwn > C exp1.c

```c
#include <stdio.h>
#include <unistd.h>

int main() {
    volatile int secret = 1234;
    char buffer[8];

    printf("Before: secret = %d\n", secret);
    printf("Enter text: ");

    read(0, buffer, 32);

    printf("After: secret = %d\n", secret);

    return 0;
}
```

```
./exp1
Before: secret = 1234
aaaa
Enter text: After: secret = 1234
```

/*WHERE THERE'S A SHELL, THERE'S A WAY*/

SHELLMATES CLUB

BUG

GDB VS

pwn > C exp1.c

```c
#include <stdio.h>
#include <unistd.h>

int main() {
    volatile int secret = 1234;
    char buffer[8];

    printf("Before: secret = %d\n", secret);
    printf("Enter text: ");

    read(0, buffer, 32);

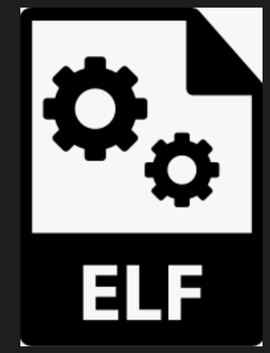
    printf("After: secret = %d\n", secret);

    return 0;
}
```

/*WHERE THERE'S A SHELL, THERE'S A WAY*/

SHELLMATES CLUB

CHANGE THE APP'S LOGIC



pwn > C exp2.c

```c
#include <stdio.h>
#include <unistd.h>

int main() {
    volatile int secret = 0;
    char buffer[8];

    printf("Enter password: ");

    read(0, buffer, 32);

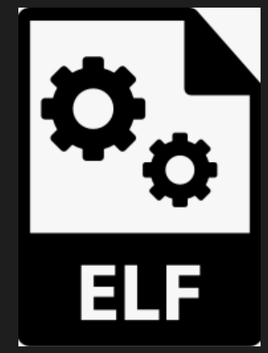
    if (secret == 1234) {
        printf("Correct password!\n");
    } else {
        printf("Wrong password!\n");
    }

    return 0;
}
```

/*WHERE THERE'S A SHELL, THERE'S A WAY*/

SHELLMATES CLUB

CHANGE THE APP'S CONTROL FLOW



pwn > C exp3.c

```c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

void win() {
    printf("You reached the WIN function!\n");
    system("/bin/sh");
}

void vuln() {
    char buffer[8];

    printf("Enter text: ");
    read(0, buffer, 64);
}

int main() {
    vuln();
    printf("Normal exit.\n");
    return 0;
}
```

/ WHERE THERE'S A SHELL, THERE'S A WAY /



SHELLMATES
CLUB

BUG CLASSES

INTEGER VULNERABILITIES

SHELLMATES
CLUB

INTEGER VULNERABILITIES

- Integer Types & Ranges

INTEGERS

- Arithmetic & Conversions

CHECK THE TYPE, RANGE, AND HOW THE RESULT IS USED.

- How the Result Is Used

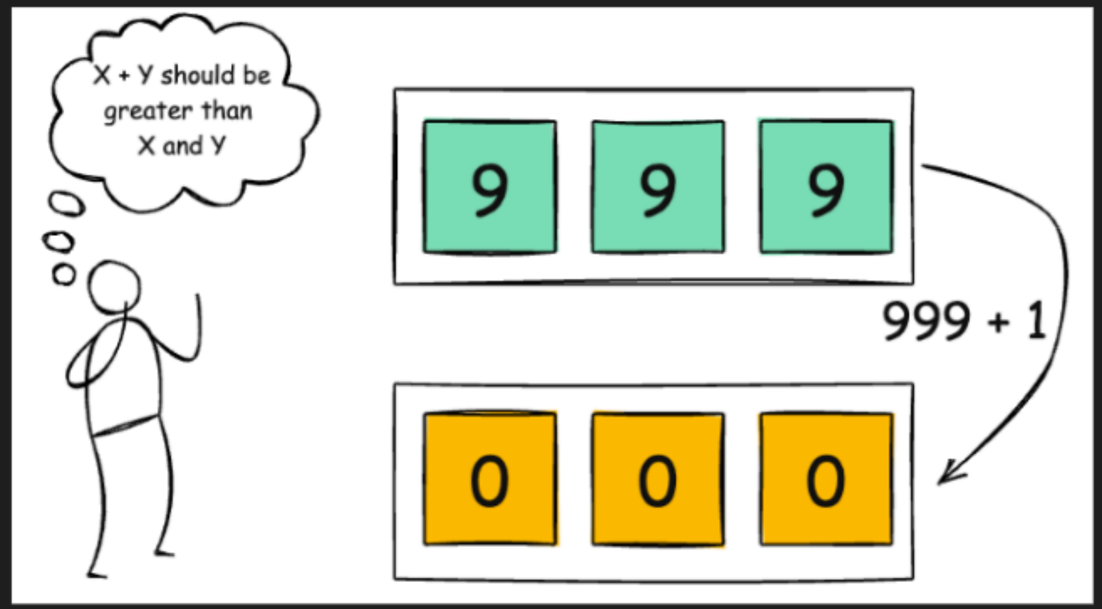
MATH OPERATIONS

/*WHERE THERE'S A SHELL, THERE'S A WAY*/

SHELLMATES CLUB

WORKED EXAMPLE

Poor understanding of the language and its data types.



/*WHERE THERE'S A SHELL, THERE'S A WAY*/

SHELLMATES CLUB

IMPACT

README License



CVE-2026-110264: GitLab Pre-Auth RCE via Onigmo

al4n4n/CVE-2026-110264



CVE-2026-110264: GitLab pre-auth RCE via Onigmo integer overflow in org-ruby. Docker-based validation, admin check script, attack surface analysis.

2 Contributors · 0 Issues · 1 Star · 0 Forks

al4n4n/CVE-2026-110264: CVE-2026-110264: GitLab pre-auth RCE via Onigmo integer overflow in org-ruby. Docker-based validation, admin…

CVE-2026-110264: GitLab pre-auth RCE via Onigmo integer overflow in org-ruby. Docker-based validation, admin check script, attack surface analysis. - al4n4n/CVE-2026-110264

GitHub

/*WHERE THERE'S A SHELL, THERE'S A WAY*/

SHELLMATES
CLUB

Q&A

SUMMARY

/*WHERE THERE’S A SHELL, THERE’S A WAY*/



SHELLMATES
CLUB

RESOURCES TO LEARN MORE

al-wasmo.github.io/blog/pwn-and-reverse-engineering-workshop#resources



SHELLMATES
CLUB

BUILD YOUR ONLINE PRESENCE

Create a blog or portfolio.

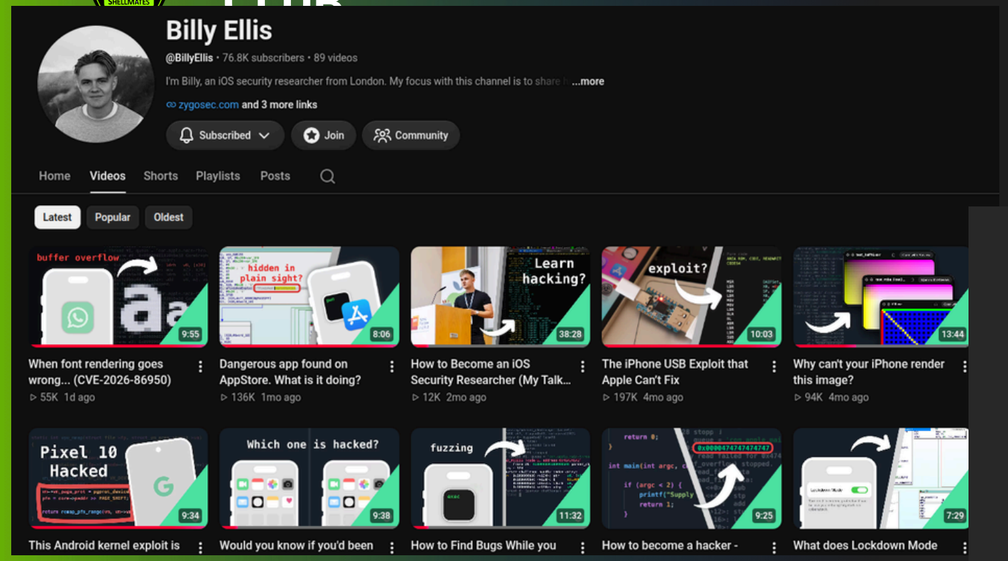
Share what you’re learning.

Document your journey.

Publish your projects and research.

/*WHERE THERE’S A SHELL, THERE’S A WAY*/

SHELLMATES CLUB





Billy Ellis

I'm Billy, an iOS security researcher from London. My focus with this channel is to share high quality technical breakdowns looking at topics including mobile security, O...

YouTube



$ WHOAMI

I'm Bahae Bahrini , AKA 'Buddurid' , 22 yo , ICT engineering student from Tunisia .

I usually play CTF's with AresX or WorldWideFlags , and CrémeTartinéFabuleuse locally .

This blog will contain some writeups of the challenges i play or author (mostly PWN stuff) .

$ CVE's

- CVE-2025-68615 in Net-SNMP

$ INTERESTS

i'm diving into these topics

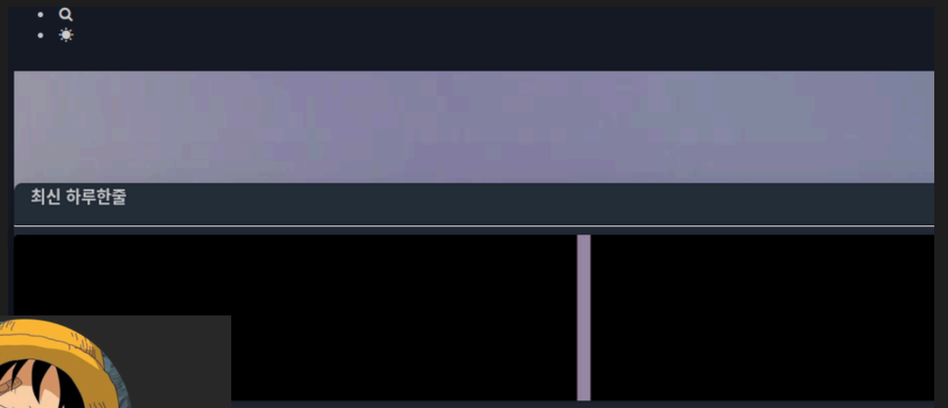
- kernel exploitation , currently exploring the linux kernel
- browser exploitation , mostly v8

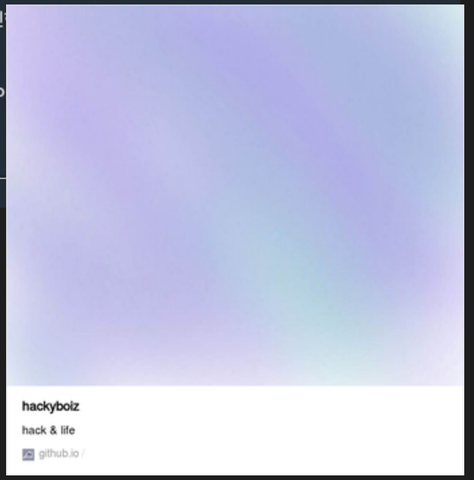


Buddurid's blog

this is a blog for my CTF's writeups (mainly PWN) and maybe my future research papers .

Buddurid's blog





/*WHERE THERE'S A SHELL, THERE'S A WAY*/



SHELLMATES
CLUB

JOB MARKET

1. Opportunities in Algeria vs. Worldwide
Limited local opportunities and interest
High global demand for specialized skills
Remote work and international opportunities

2. Bug Bounty
Google VRP (Chrome, Android, etc.)
Finding vulnerabilities in software and libraries
Getting rewarded for security discoveries

SHELLMATES CLUB

JOB MARKET

3. Vulnerability Research & R&D
Zero-day and one-day vulnerability research
Browser, kernel, mobile, and hardware security
Research labs and specialized security companies

4. Teaching & Training
Cybersecurity workshops and courses
Advanced reverse engineering and exploitation training
Corporate and government security training

SHELLMATES CLUB

THANK YOU FOR YOUR ATTENTION!

/*WHERE THERE'S A SHELL, THERE'S A WAY*/